Pràctica 2. Creació i duplicació de processos.

Primer programa en C

Si hi ha un programa obligatori a la hora de començar a programar en un llenguatge de programació, aqueix és el mític “Hola, món!”.

```
#include <stdio.h>
int main( int argc, char *argv[] )
{
        printf( "Hola, món!\n");
        return 0;
}
```

Per a compilar un programa en C mitjançant el compilador gcc necessitarem l'ordre:

```
mint@mint:~$ gcc T1S2Exemple1HolaMon.c -o T1S2Exemple1HolaMon
```

L'ordre anterior compila el programa en C hola.c, genera un arxiu executable hola. Ara executem el programa i la línia a continuació és l’eixida.

```
mint@mint:~$ ./T1S2Exemple1HolaMon
Hola, món!
```

Depenent de la distribució de Linux, potser faça falta instal·lar alguna llibreria. Prova amb:

```
mint@mint:~$ sudo aptitude install libc-dev g++ build-essential
```

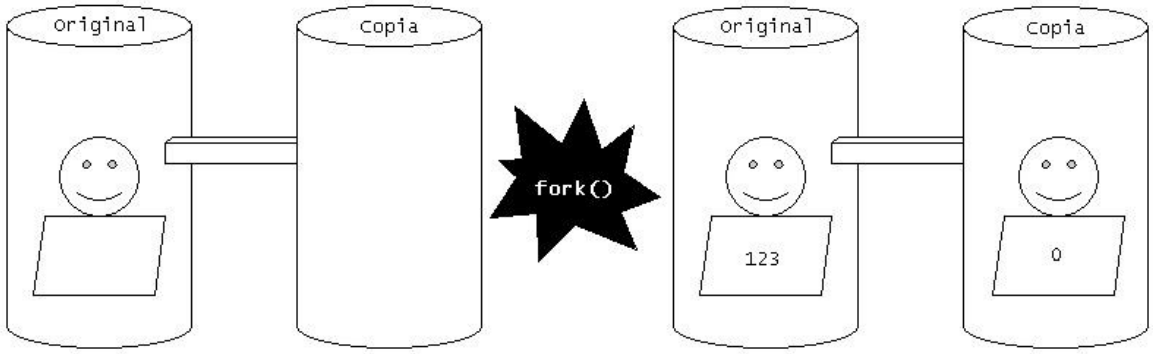
Creació i duplicació de processos.

Una situació molt habitual dins d’un programa és la de crear un nou procés que s'encarregue d'una tasca concreta, descarregant al procés principal de tasques secundàries que poden realitzar-se asíncronament o en paral·lel. Linux ofereix diverses funcions per a realitzar això: system(), fork() i exec().

Ens centrem en primer lloc en fork(). Aquesta funció crea un procés nou o procés fill que és exactament igual que el procés pare. Si fork( ) s'executa amb èxit retorna:

- Al pare: el PID del procés fill creat.
- Al fill: el valor 0.

Per a entendre'ns, fork() clona els processos (realment és clone() qui clona els processos, però fork() fa una cosa bastant similar). És com una màquina per a replicar persones: en una de les dues cabines de la nostra màquina entra una persona. S'activa la màquina i aqueixa persona és clonada. En la cabina contigua hi ha una persona idèntica a la primera, amb els seus mateixos records, mateixa edat, mateix aspecte, etc.



Per tant, quan comença el programa, només hi ha un procés: el procés original. Quan el procés executa la instrucció fork(), el sistema crea un altre procés. Tindrem el procés pare (original) i el procés fill. Els dos tenen el mateix programa associat i, per tant, el mateix codi. Els dos tindran el record d'haver executat el programa fins el fork però cadascú s'executarà de manera independent a partir de la següent instrucció de fork.

Normalment, m'interessarà que el pare faça coses diferents a la del fill. Per tant, després del fork m'interessa diferenciar el codi del programa en dues parts: una per al pare i una altra per al fill. La manera de fer-ho és mitjançant una instrucció selectiva if-else. La qüestió és saber com puc fer per a que el pare sàpiga que és el pare i el fill el fill.

Quan executem fork, el sistema torna un nombre enter que permet aqueixa diferenciació. En el cas de ser el fill, fork torna el valor 0. En el cas de ser el pare, fork torna el PID del procés fill creat.

Fixa't en el codi (T1S2Exemple2BasicMon.c):

```
#include <sys/types.h>
#include <unistd.h>
#include <stdio.h>
#include <sys/wait.h>
int main(int argc, char *argv[])
{
        pid_t pid;
        pid=fork();
        if ( pid == 0 )
        { /* fill */
                printf("Soc el fill (%d, fill de %d)\n", getpid(), getppid());
        }
        else
        { /* pare */
                printf("Soc el pare (%d, fill de %d)\n", getpid(), getppid());
        }
        return 0;
}
```

Guardem en la variable pid el resultat de fork() Si és 0, resulta que estem en el procés fill, per la qual cosa farem el que haja de fer el fill. Si és diferent de zero, estem dins del procés pare, per tant tot el codi que vaja en la part “else” d'aqueixa instrucció condicional només s'executarà en el procés pare.

L'eixida de l'execució d'aquest programa és la següent:

```
mint@mint:~$ gcc T1S2Exemple2BasicMon.c -o T1S2Exemple2BasicMon
mint@mint:~$ ./T1S2Exemple2BasicMon
Soc el pare (569, fill de 314)
Soc el fill (570, fill de 569)
mint@mint:~$ pgrep bash
314
```

L'eixida de les dues crides a printf(), la del pare i la del fill, són asíncrones, és a dir, podria haver eixit primer la del fill, ja que està corrent en un procés separat, que pot executar-se abans en un entorn multiprogramat. El fill, 570, afirma ser fill de 569, i el seu pare, 569, és al seu torn fill de la shell en la qual ens trobem, 314.

Això provoca que potser acaba abans el pare que el fill, provocant que quan el fill pregunte qui és el seu pare amb la instrucció getppid(), al no tenir pare (pq ha acabat), li conteste un altre PID. Quan passa això, fixat com l’eixida per la terminal està mal formatada i hi ha que donar-li ENTER per a que torne a eixir el prompt.

Si volguérem que el pare esperara a algun dels seus fills haurem de dotar de sincronisme a aquest programa, utilitzant les següents funcions:

```
pid_t wait(int *status)
pid_t waitpid(pid_t pid, int *status, int options);
```

La primera d'elles espera a qualsevol dels fills i retorna en la variable sencera status l'estat d'eixida del fill (si el fill ha acabat la seua execució sense error, el que és normal és que haja retornat zero). La segona funció, waitpid(), espera a un fill en concret, el que especifiquem en pid. Aqueix PID o identificatiu de procés ho obtindrem en fer la crida a fork() per a aqueix fill en concret, per la qual cosa convé guardar el valor retornat per fork(). En el següent exemple combinarem la crida a waitpid() amb la creació d'un arbre de processos més complex, amb un pare i dos fills (T1S2Exemple3DosFills.c):

```
#include <sys/types.h>
#include <unistd.h>
#include <stdio.h>
#include <sys/wait.h>
```

```
int main(int argc, char *argv[])
{
	pid_t pid1, pid2;
	int status1, status2;
	pid1=fork();
	if ( pid1== 0 )
	{ /* fill */
		printf("Soc el primer fill (%d, fill de %d )\n", getpid(), getppid());
	}
	else
	{ /* pare */
		pid2=fork();
		if ( pid2 == 0 )
		{ /* segon fill */
			printf("Soc el segon fill (%d, fill de %d )\n", getpid(), getppid());
		}
		else
		{ /* pare */
		/* Esperem al primer fill acabe, status 0*/
			waitpid(pid1, &status1, 0);
		/* Esperem al segon fill acabe, status 0 */
			waitpid(pid2, &status2, 0);
			printf("Soc el pare (%d, fill de %d)\n", getpid(), getppid());
		}
	}
	return 0;
}
```

El resultat de la execució d'aquest programa és aquest:

```
mint@mint:~$ gcc T1S2Exemple3DosFills.c -o T1S2Exemple3DosFills
mint@mint:~$ ./T1S2Exemple3DosFills
Soc el primer fill (15503, fill de 15502)
Soc el segon fill (15504, fill de 15502)
Soc el pare (15502, fill de 15471)
mint@mint:~$ pgrep bash
15471
```

Amb waitpid assegurem que el pare esperarà als seus dos fills abans de continuar, per la qual cosa el missatge de “Soc el pare” sempre eixirà l'últim.
Fixa’t que encara utilitzant wait o waitpid, no podem asegurar quin dels dos fills acabarà abans. De fet, si ho executes múltiples vegades voràs que hi ha vegades que acaba primer el primer fill i d’altres el segon.

T1S2P2TresGeneracions.c

Avi --- fork () --> Fill --- fork() --> Net

Realitza un programa anomenat UD1P1TresGeneracions.c en el qual l'arbre cobrisca tres generacions i l'eixida ha de ser sempre tindrà l'ordre: net, pare, avi. L'eixida en executar-ho ha de ser la següent:

```
mint@mint:~$ gcc T1S2P2TresGeneracions.c -o T1S2P2TresGeneracions
mint@mint:~$ ./T1S2P2TresGeneracions
Soc el net (15565, fill de 15564)
Soc el pare (15564, fill de 15563)
Soc l'avi (15563, fill de 15471)
mint@mint:~$ pgrep bash
15471
```

T1S2P2Variable.c

Realitza un programa en C anomenat T1S2P2Variable.c que cree un procés (tindrem 2 processos: un pare i un fill). El programa definirà una variable sencera i li donarà el valor 6. El procés pare incrementarà aquest valor en 5 i el fill restarà 5. S'han de mostrar els valors en pantalla. A continuació es mostra exemple de l'execució.

```
mint@mint:~$ gcc T1S2P2Variable.c -o T1S2P2Variable
mint@mint:~$ ./T1S2P2Variable
Soc el Pare. Valor variable és 11
Soc el Fill. Valor variable és 1
```

Explica el resultat del programa (recorda els canvis de contextos explicats en la teoria).